reducing cache misses and page swaps have been proposed over the years: we will talk about splay trees in chapter 7.

While these alternatives aren't in general able to have the same balance between practical and theoretical performance as heaps, there ought to say that when the cost of page faults or disk access dominates, it might be better to resort to a linearithmic[23] algorithm with higher locality rather than sticking with a linear algorithm with poor locality.

2.10 Performance Analysis: Finding the best Branching Factor

So far for the theory: now let’s try to apply it to a real case, and doing so describe how to profile the implementation of a data structure, and in general an application.

We have seen that priority queues are a key component in the Huffman compression pipeline; if we measure the performance of our heap’s methods as the number of swaps performed, we have shown that we have at most `h` swaps per-method call, if `h` is the height of the heap. In section 2.5 we have also shown that, because the heap is a *complete balanced tree*, the height of a d-ary heap is exactly `logD(n)` [24].

Therefore, for both methods insert and top, it would seem that the larger the branching factor, the smaller the height, and in turn the better heap performance should be.